

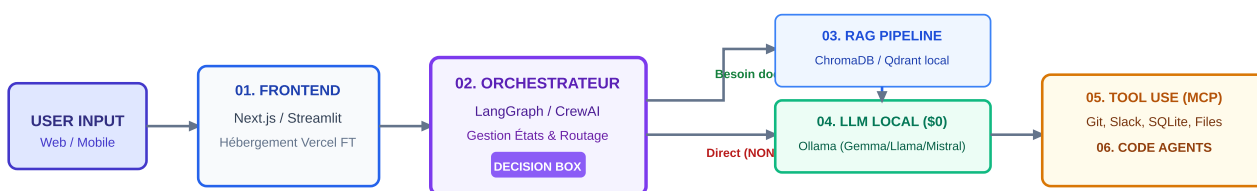
The \$0 AI Architecture Stack

Conception, orchestration et déploiement d'applications autonomes d'entreprise sans coût de licence propriétaire ni dépendance cloud payante.

1. Vue d'Ensemble & Paradigme Zéro Dollar

L'écosystème de l'intelligence artificielle en 2026 permet de concevoir des systèmes de production complets sans déboursier le moindre centime en abonnements API ou en hébergements premium. En s'appuyant sur des modèles open-weights exécutés en local, des protocoles standardisés (MCP), des orchestrateurs à graphe d'états et les paliers gratuits (Free Tiers) des plateformes cloud, les ingénieurs peuvent déployer des architectures robustes et souveraines.

SCHÉMA FONCTIONNEL GLOBAL DE LA STACK \$0 (FLUX SYNCHRONE & ASYNCHRONE)



2. Couche 01 — Frontend Layer : Routage et Interface Utilisateur

Next.js + Vercel (Free Tier)

PRODUCTION WEB

Permet de bâtir des interfaces riches avec support de streaming token-by-token (via SSE ou WebSocket).

Le palier gratuit de Vercel supporte les fonctions Serverless, les Edge Functions et le cache CDN mondial sans engagement financier.

Streamlit (Python Native)

PROTOTYPAGE / DATA

La solution idéale pour les ingénieurs Data/IA souhaitant coder l'UI en pur Python. Compatible avec l'hébergement Streamlit Community Cloud ou un conteneur local, il réduit le temps de mise sur le marché d'une interface d'inférence à quelques heures.

3. Couche 02 — Agent Orchestrator : Le Cerveau Systémique

L'orchestrateur gouverne le cycle de vie de la requête utilisateur. Contrairement aux chaînes linéaires simplistes, l'architecture 2026 repose sur des **machines à états cycliques** capables d'itérer, d'évaluer les résultats intermédiaires et de décider d'un retour en arrière (re-planning).

LangGraph

GRAPHES D'ÉTATS DIRIGÉS

Implémente des architectures agenciques sous forme de StateGraph. Permet le streaming d'états partagés, la persistance fine et l'exécution conditionnelle déterministe (routage selon la classification de la requête).

CrewAI

SYSTÈMES MULTI-AGENTS

Organise la division du travail entre plusieurs agents spécialisés (Researcher, Writer, Reviewer). Chaque agent possède un rôle strict, des outils dédiés et une mémoire partagée pour simuler une équipe opérationnelle.

4. Couche 03 — RAG Pipeline : Connaissances & Vector Stores

Le composant RAG (Retrieval-Augmented Generation) s'active dès que le système requiert des données privées ou fraîches. Pour maintenir le coût à 0\$, la pipeline s'appuie sur des connecteurs documentaires ouverts et des bases vectorielles embarquées.

Notion API

SOURCE

Utilisé pour planifier et indexer la base de connaissances métier via des intégrations gratuites.

ChromaDB

IN-MEMORY

Base vectorielle open source ultra-légère, idéale en local ou en conteneur éphémère.

Qdrant (Local)

HAUTE PERF

Moteur vectoriel écrit en Rust, exécutable via Docker avec support du filtrage hybride (dense + BM25).

Mécanisme de Routage Conditionnel : Le graphe évalue si la requête nécessite des connaissances externes. Si `Need external knowledge? == YES`, la question est transformée en requête vectorielle (embedding local via *nomic-embed-text* ou *bge-small*), le contexte pertinent est extrait de Qdrant/ChromaDB et injecté dans le prompt système du modèle.

5. Couche 04 — LLM Layer : L'Inférence Locale Souveraine (\$0)

En 2026, l'inférence locale haute performance via **Ollama** est le pilier financier de cette architecture. Elle élimine la facturation par token tout en garantissant la confidentialité des données traitées.

Modèle & Runtime	Empreinte Matérielle	Cas d'Usage Optimaux	Forces & Spécificités
Gemma 4 E4B (via Ollama runtime)	Légère (4 à 8 Go VRAM / RAM unifiée)	Classification, routage d'orchestration, tâches légères de filtrage.	Latence ultra-faible (>75 tok/s sur puce Apple ou RTX 4060). Idéal pour la prise de décision rapide.
Mistral Small 4 (Mistral AI Open-Weights)	Intermédiaire (12 à 16 Go VRAM)	Raisonnement analytique, synthèse multilingue, appels de fonctions.	Excellente performance en français, respect strict du formatage JSON et du function calling.
Llama 3.3 70B (Meta AI - Quantifié Q4_K_M)	Exigeante (24 à 40 Go VRAM ou Mac Studio M-series)	Raisonnement complexe, synthèse de dossiers lourds, génération logicielle.	Rivalise directement avec les modèles propriétaires de pointe de classe GPT-4o / Claude Sonnet sans frais d'API.

Pourquoi Ollama domine la stack \$0 ?

Ollama abstrait la gestion de la compilation GPU (Metal, CUDA, ROCm), gère le swap dynamique de modèles en mémoire et expose une API compatible avec le protocole OpenAI (port 11434), rendant la bascule transparente pour le frontend et l'orchestrateur.

6. Couche 05 — Tool Use via MCP (Model Context Protocol)

Le **Model Context Protocol (MCP)** est devenu le standard industriel unifié permettant aux LLMs d'interagir avec des ressources externes sans créer de connecteurs ad-hoc fragiles.

Rôle & Avantages de MCP

STANDARD OUVERT

MCP standardise l'exposition d'outils (Tools), d'invites (Prompts) et de ressources (Resources). Le LLM reçoit le schéma typé des outils disponibles et formule des requêtes d'exécution standardisées que l'hôte exécute en toute sécurité.

Serveurs MCP Types (\$0)

ÉCOSYSTÈME

- **GitHub MCP** : Navigation repos, commits, lecture de pull requests.
- **Slack / Discord MCP** : Envoi de notifications et réception d'alertes.
- **Filesystem & SQLite MCP** : Manipulation sécurisée de données locales.

7. Couche 06 — Code Agent : Ingénierie Logicielle Assistée

Les agents de codage ne se limitent plus à la complétion : ils écrivent, testent, corrigent et déploient du code de manière autonome en accédant directement au terminal et à l'arborescence des projets.

Claude Code CLI

AGENT TERMINAL

Outil d'ingénierie en ligne de commande opérant sur l'ensemble du dépôt logiciel. Capable de naviguer dans l'arborescence, d'éditer plusieurs fichiers simultanément et d'exécuter la suite de tests automatisés.

Aider (Open Source)

PAIR PROGRAMMING GIT

Agent terminal 100% compatible avec les modèles locaux d'Ollama. Il commite automatiquement chaque bloc de modification avec des messages Git explicites et gère les refactorings complexes.

8. Couche 07 — Data Layer : Persistance & Gestion d'État

Pour garantir la persistance des sessions et des états d'application sans coût récurrent, trois moteurs complémentaires constituent la couche de stockage :

SQLite

Base de données relationnelle embarquée. Aucun serveur requis. Idéale pour stocker l'historique conversationnel et l'état applicatif local.

DuckDB

Moteur OLAP vectorisé en mémoire. Permet d'effectuer des requêtes SQL analytiques ultra-rapides sur des millions de logs ou fichiers Parquet.

Supabase (Free)

PostgreSQL infogéré avec pgvector, authentification et base temps réel dans les quotas généreux du palier gratuit.

Coordination de l'État : L'état de l'orchestrateur (Application State) est synchronisé en continu avec la base de données relationnelle pour permettre le redémarrage sans perte en cas de défaillance (Checkpointing).

9. Observabilité — Phoenix (Arize) Auto-Hébergé

L'observabilité est obligatoire pour auditer les hallucinations, tracer les appels d'outils et profiler la latence. **Arize Phoenix** est déployé localement sans coût de licence.

Traçage OpenTelemetry

INSTRUMENTATION

Capture chaque nœud du graphe LangGraph : latence de réponse du LLM local, tokens générés, requêtes vectorielles exactes envoyées à Qdrant.

Évaluation & Evals Continues

AUDIT QUALITÉ

Permet de calculer automatiquement la fidélité documentaire (Groundedness), la pertinence du contexte extrait et la détection d'erreurs de syntaxe MCP.

10. Comparatif Économique : Stack Propriétaire vs Stack \$0 (2026)

Composant Système	Architecture Commerciale Traditionnelle	Architecture \$0 (Cours 2026)
Inférence LLM	OpenAI / Anthropic API (~150€ à 800€/mois)	Ollama en local (Llama 3.3, Mistral, Gemma) : 0€
Base Vectorielle	Pinecone Standard (~70€/mois)	Qdrant Docker / ChromaDB : 0€
Hébergement Frontend	AWS / GCP dédié (~40€/mois)	Vercel Free Tier / Streamlit Cloud : 0€
Observabilité IA	Datadog LLM / LangSmith Pro (~90€/mois)	Phoenix Open-Source Auto-hébergé : 0€
Persistance & État	Bases DBaaS gérées (~30€/mois)	SQLite / DuckDB / Supabase Free Tier : 0€

11. Synthèse Méthodologique de Déploiement

Pour mettre en œuvre avec succès cette architecture en environnement réel, suivez la séquence d'intégration suivante :

- Étape 1 (Calcul & Inférence)** : Installer Ollama, configurer l'accélération matérielle GPU et télécharger les poids des modèles (*gemma4:e4b* pour le routage et *llama3.3:70b* pour l'analyse).
- Étape 2 (Stockage Vectoriel & RAG)** : Lancer Qdrant en local via un simple conteneur, initialiser l'index avec une distance cosinus et connecter vos notebooks de parsing.
- Étape 3 (Moteur d'Orchestration)** : Développer le StateGraph LangGraph reliant le routage conditionnel, la récupération de contexte et l'exécution MCP.
- Étape 4 (Monitoring)** : Activer l'instrumentation Phoenix sur le port local 6006 pour tracer l'intégralité des flux dès la première requête d'essai.

Conclusion : La pile \$0 n'est pas une architecture dégradée : elle procure une maîtrise technique totale, protège le secret industriel sans fuite de données et offre une marge brute maximale pour l'ingénieur ou la startup.